

PYTHON MASTER NOTES

PART 3

Conditions, Loops & Functions

A beginner-friendly, interview-level study guide with worked examples, diagrams, traps, cheat sheets and 90 practice questions with complete solutions.

Marker	Meaning
★ MUST KNOW	Core idea. If you skip this, everything later gets harder.
⊙ INTERVIEW	Frequently asked, or a favourite trick question.
⚠ COMMON MISTAKE	A bug beginners (and tired experts) write all the time.
⚡ SHORTCUT	A faster or more Pythonic way of writing it.
◆ DEEP CONCEPT	What Python is really doing under the hood.

Every code sample marked with an **Output** divider was executed while this PDF was generated, so the printed output is real, not guessed. All 30 coding solutions at the end were also run against tests.

Contents

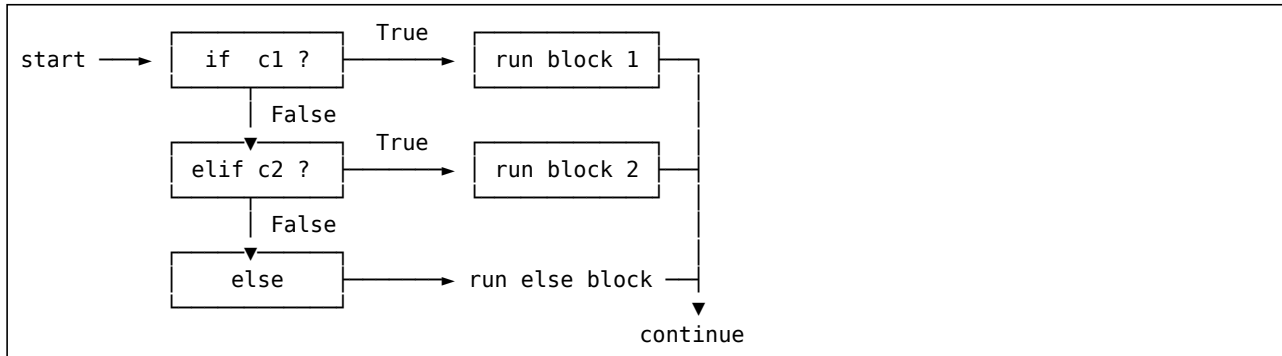
Chapter 1 · Conditions	4
1.1 if, elif, else	4
1.2 Nested conditions and guard clauses	5
1.3 Conditional expression (ternary operator)	5
1.4 Truthy and falsy	6
1.5 Chained comparisons	6
1.6 and, or, not	7
1.7 Common condition mistakes	8
Chapter 2 · Loops	9
2.1 The for loop	9
2.2 range()	9
2.3 The while loop	10
2.4 Nested loops	10
2.5 break, continue, pass	11
2.6 Loop else	11
2.7 Iterating over strings, lists and dictionaries	12
2.8 enumerate() and zip()	13
2.9 Common loop patterns	13
Chapter 3 · Functions	15
3.1 Defining and calling	15
3.2 Parameters vs arguments	15
3.3 return, multiple return values, return vs print	16
3.4 Positional, keyword and default arguments	17
3.5 Variable-length arguments: *args and **kwargs	17
3.6 Positional-only and keyword-only parameters	18
3.7 The mutable default argument trap	18
3.8 Lambda functions	19
3.9 First-class functions and functions as objects	20
3.10 Recursion	20
3.11 Scope: local, global, LEGB	21
Chapter 4 · Concept Clinic: Side-by-Side	24
4.1 Parameter vs argument	24
4.2 return vs print	24
4.3 Local vs global	24

4.3b Recursion vs iteration	24
4.4 Lambda vs def	24
4.5 First-class functions and functions as objects	25
4.6 Mutable default argument trap (one-line summary)	25
4.7 == vs is, and args vs kwargs	25
Chapter 5 · Real Coding Examples	26
5.1 FizzBuzz	26
5.2 Prime check and GCD	26
5.3 Palindrome and vowel count	26
5.4 Binary search (function + while loop)	26
5.5 Calculator with a dispatch table	27
5.6 Flatten a nested list (recursion)	27
5.7 Password validator (conditions + loops + any/all)	27
5.8 Function with *args, **kwargs and keyword-only settings	27
Chapter 6 · Revision Notes	28
Conditions in one page	28
Loops in one page	28
Functions in one page	28
Chapter 7 · Function Cheat Sheet	29
Argument-order diagram	29
Decision guide	30
Chapter 8 · Practice Questions	31
Part A · 30 Conceptual Questions	31
Part B · 30 Output Questions	32
Part C · 30 Coding Questions	35
Chapter 9 · Complete Solutions	36
Part A · Conceptual Answers	36
Part B · Output Answers	38
Part C · Coding Solutions	40
Chapter 10 · Common Interview Traps	45
Last-minute checklist	46

Chapter 1 · Conditions

A **condition** lets a program choose between paths. Python checks conditions from top to bottom and runs only the **first** block whose condition is truthy. Everything else is skipped.

1.1 if, elif, else



- if starts the chain. elif (else-if) is optional and can repeat. else is optional and always last.
- A colon : ends each header line. The **indented block** under it is the body (use 4 spaces).
- The condition can be any expression. Python converts it to True/False (see truthy/falsy, 1.4).

```
marks = 82
if marks >= 90:
    print("A")
elif marks >= 75:
    print("B")
elif marks >= 50:
    print("C")
else:
    print("Fail")
```

Output _____
B

★ MUST KNOW

Only ONE branch of an if/elif/else chain runs. As soon as one condition is true, the rest are skipped. Two separate if statements are independent and BOTH can run.

```
x = 10
if x > 5: print("if-1")
if x > 8: print("if-2")          # separate if: also runs
print("--- with elif ---")
if x > 5: print("first")
elif x > 8: print("never reached")
```

Output _____
if-1
if-2
--- with elif ---
first

⚠ COMMON MISTAKE

Wrong order of elif. if marks >= 50 before if marks >= 90 means a 95 prints "C". Always put the most specific / highest condition first.

1.2 Nested conditions and guard clauses

An if inside another if is a **nested condition**. Deep nesting gets hard to read, so prefer flat code using and, or exit early (a **guard clause**).

```
age, has_id = 20, True
# nested
if age >= 18:
    if has_id:
        print("Entry allowed")
# flat: same meaning
if age >= 18 and has_id:
    print("Entry allowed (flat)")
_____ Output _____
Entry allowed
Entry allowed (flat)
```

```
def can_vote(age, citizen):
    if age < 18:          # guard clauses: reject early
        return "too young"
    if not citizen:
        return "not a citizen"
    return "can vote"    # the happy path stays un-nested
print(can_vote(17, True), "|", can_vote(30, False), "|", can_vote(30, True))
_____ Output _____
too young | not a citizen | can vote
```

✂ SHORTCUT

Guard clauses (return / continue early on bad input) keep the main logic at one indent level. Interviewers like this style.

1.3 Conditional expression (ternary operator)

Python's ternary is written ****value_if_true if condition else value_if_false****. Note the order: the value comes first, the condition in the middle.

```
x = -3
result = "positive" if x > 0 else "non-positive"
print(result)
print("even" if 8 % 2 == 0 else "odd")
# nested ternary (legal but hard to read, avoid beyond one level)
n = 0
print("pos" if n > 0 else "neg" if n < 0 else "zero")
_____ Output _____
non-positive
even
zero
```

© INTERVIEW

Ternary inside a comprehension goes **BEFORE** for (it transforms). A plain if filter goes **AFTER** for (it filters). [x if x>0 else 0 for x in a] vs [x for x in a if x>0].

```
a = [3, -1, 4, -5]
print([x if x > 0 else 0 for x in a])    # transform: keeps length
print([x for x in a if x > 0])          # filter: drops items
_____ Output _____
[3, 0, 4, 0]
[3, 4]
```

Bonus: walrus operator := (assignment inside an expression)

```
data = [4, 8, 15, 16, 23]
if (n := len(data)) > 3:
    print("long list of", n)
_____ Output _____
long list of 5
```

1.4 Truthy and falsy

Any object can be used as a condition. Python calls `bool(obj)` for you. Objects that count as **empty or zero** are falsy; almost everything else is truthy.

Falsy (evaluates to False)	Truthy (evaluates to True)
False, None	True, any non-zero number (-1, 3.5)
0, 0.0, 0j	non-empty string, even "0" and "False"
"" (empty string)	non-empty list/tuple/set/dict, even [0] or [[]]
[], (), {}, set(), range(0)	most user-defined objects (unless they define <code>__bool__</code> / <code>__len__</code>)

```
print(bool(0), bool(""), bool([]), bool(None))
print(bool("0"), bool("False"), bool([0]), bool(-1))
arr = []
if not arr:
    print("list is empty")
_____ Output _____
False False False False
True True True True
list is empty
```

★ MUST KNOW	<code>if arr:</code> means "arr is not empty" and <code>if not arr:</code> means "arr is empty". This replaces <code>if len(arr) > 0</code> . Use it.
⚠ COMMON MISTAKE	<code>if x:</code> is WRONG when 0 or "" is a valid value. If x can legitimately be 0, test <code>if x is not None:</code> . Example: <code>if score:</code> skips a real score of 0.
◆ DEEP CONCEPT	For your own class, <code>bool(obj)</code> calls <code>obj.__bool__()</code> ; if that is missing it calls <code>obj.__len__()</code> (0 is falsy); if both are missing the object is always truthy.

1.5 Chained comparisons

Python lets you chain comparisons the way mathematics does. `a < b < c` means `a < b` and `b < c`, and `b` is evaluated only once.

```
x = 7
print(1 < x < 10)      # True: like 1 < x and x < 10
print(1 < x < 5)       # False
print(3 == 3 == 3)    # True
print(1 < 2 > 1)       # True: (1<2) and (2>1)
print(1 == 1 < 2)     # True
_____ Output _____
True
False
True
True
True
```

⚠ COMMON MISTAKE	Chaining works with <code>==</code> and <code><</code> mixing, but <code>a == b == c</code> does NOT mean "a equals b or c". Also the beginner bug <code>x == 1 or 2</code> is always truthy because 2 alone is truthy. Write <code>x == 1 or x == 2</code> , or better <code>x in (1, 2)</code> .
-------------------------	---

```
x = 5
print(x == 1 or 2)     # prints 2, the "or" returns its second operand
print(x in (1, 2))    # correct membership test
_____ Output _____
2
False
```

1.6 and, or, not

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Precedence (high to low): comparisons, then not, then and, then or. Use parentheses whenever unsure.

Short-circuit evaluation: A and B stops at A if A is falsy. A or B stops at A if A is truthy. The right side is not even evaluated.

```
def loud(v):
    print("evaluating", v); return v
print(loud(0) and loud(5))    # stops at 0
print(loud(3) or loud(9))    # stops at 3
b = 0
print(b != 0 and 10 / b > 1)  # safe: division never happens
_____ Output _____
evaluating 0
0
evaluating 3
3
False
```

◆ DEEP CONCEPT

and / or do NOT return True/False. They return one of the operands. A or B returns the first truthy operand (else the last). A and B returns the first falsy operand (else the last).

```
print(0 or "" or "fallback")  # first truthy
print(1 and 2 and 3)         # all truthy -> last
print(1 and 0 and 3)         # first falsy
name = ""
print(name or "Guest")       # default-value idiom
print(True or False and False) # and binds tighter
print((True or False) and False)
print(not 1 == 2)            # not (1 == 2)
_____ Output _____
fallback
3
0
Guest
True
False
True
```

⚡ SHORTCUT

value = user_input or default gives a default when the input is empty/None/0. Careful: 0 counts as empty here.

1.7 Common condition mistakes

Mistake	Why it is wrong	Fix
<code>if x = 5:</code>	<code>=</code> is assignment: <code>SyntaxError</code>	<code>if x == 5:</code>
<code>if x == 1 or 2:</code>	2 is always truthy	<code>if x in (1, 2):</code>
<code>if x == None:</code>	works but un-Pythonic; <code>==</code> can be overridden	<code>if x is None:</code>
<code>if flag == True:</code>	redundant, and fails for truthy non-bools	<code>if flag:</code>
<code>if len(a) > 0:</code>	verbose	<code>if a:</code>
<code>if a == b == c</code> meaning "a is b or c"	chain means all equal	<code>if a in (b, c)</code>
<code>0.1 + 0.2 == 0.3</code>	float rounding gives False	<code>math.isclose(a, b)</code>
<code>if s is "hi"</code>	<code>is</code> checks identity, not value	<code>if s == "hi"</code>
missing <code>:</code> or bad indent	<code>SyntaxError</code> / <code>IndentationError</code>	colon + 4 spaces
<code>if a: ... if b: ... else:</code> when you meant a chain	<code>else</code> binds only to the last <code>if</code>	use <code>elif</code>

```
print(0.1 + 0.2 == 0.3)
import math
print(math.isclose(0.1 + 0.2, 0.3))
_____ Output _____
False
True
```

© INTERVIEW

Trap: `if x is 5` can appear to work for small ints because CPython caches them, then break for big numbers. `is` is for identity (None, True, sentinels). `==` is for value.

Bonus: match statement (Python 3.10+)

A cleaner replacement for long `if/elif` chains that compare one value. Recognise it; you will meet it in modern code.

```
def http(code):
    match code:
        case 200: return "OK"
        case 404: return "Not Found"
        case 500 | 502: return "Server error"
        case _: return "Unknown"
print(http(200), http(502), http(999))
_____ Output _____
OK Server error Unknown
```

Chapter 2 · Loops

A **loop** repeats a block of code. Python has two: **for** (repeat over a sequence of items) and **while** (repeat as long as a condition is true).

2.1 The for loop

for walks over any **iterable** (list, string, tuple, dict, set, range, file...) and hands you one item per round. Python takes care of the counter for you.

```
for fruit in ["apple", "kiwi", "fig"]:  
    print(fruit)  
for i in range(3):  
    print("round", i)
```

Output

```
apple  
kiwi  
fig  
round 0  
round 1  
round 2
```

iterable: [10 , 20 , 30]

```
round 1 ————— x = 10 → body runs  
round 2 ————— x = 20 → body runs  
round 3 ————— x = 30 → body runs → loop ends
```

⚠ COMMON MISTAKE

The loop variable is NOT deleted after the loop. `for i in range(3):` pass then `print(i)` prints 2. If the iterable is empty, the variable is never created and using it raises `NameError`.

2.2 range()

Form	Produces	Example → values
<code>range(stop)</code>	0 up to stop-1	<code>range(4)</code> → 0 1 2 3
<code>range(start, stop)</code>	start up to stop-1	<code>range(2, 6)</code> → 2 3 4 5
<code>range(start, stop, step)</code>	every step-th number	<code>range(0, 10, 3)</code> → 0 3 6 9
negative step	counts down; start must be > stop	<code>range(5, 0, -2)</code> → 5 3 1

```
range(2, 10, 3)    2  3  4  5  6  7  8  9 10  
●                ●      ●      x ← stop is EXCLUDED  
2                5      8
```

```
print(list(range(5)), list(range(2, 6)), list(range(10, 0, -3)))  
print(list(range(5, 1)))          # empty: wrong direction  
r = range(1_000_000_000)  
print(len(r), r[5], 999 in r)     # instant, nothing stored  
Output  
[0, 1, 2, 3, 4] [2, 3, 4, 5] [10, 7, 4, 1]  
[]  
1000000000 5 True
```

◆ DEEP CONCEPT

`range` is a lazy sequence object storing only start, stop, step. It uses $O(1)$ memory, supports `len`, indexing, slicing and $O(1)$ in. In Python 3 it is not a list; wrap with `list(range(...))` only when you need one.

⚠ COMMON MISTAKE	Off-by-one: <code>range(1, n)</code> stops at <code>n-1</code> . To include <code>n</code> , write <code>range(1, n + 1)</code> .
⚡ SHORTCUT	Backwards: <code>for i in range(n - 1, -1, -1)</code> or simply <code>for x in reversed(arr)</code> .

2.3 The while loop

Use `while` when you do not know in advance how many rounds you need: reading until a condition changes, digit processing, two pointers, binary search, simulations.

```
n, count = 1234, 0
while n > 0:          # count digits
    n //= 10
    count += 1
print(count)
_____ Output _____
4
```

```
# while True + break: a very common pattern
total, i = 0, 1
while True:
    total += i
    if total > 20:
        break
    i += 1
print(i, total)
_____ Output _____
6 21
```

⚠ COMMON MISTAKE	Infinite loop: forgetting to update the variable in the condition (<code>while i < 5:</code> without <code>i += 1</code>). Also <code>continue</code> placed before the increment can skip the update and loop forever.
★ MUST KNOW	Rule of thumb: known number of rounds or a collection to walk → <code>for</code> . Repeat until something happens → <code>while</code> .

2.4 Nested loops

A loop inside a loop. The inner loop runs completely for every single round of the outer loop. Two loops of size `n` each means about $n \times n$ work, written **$O(n^2)$** .

```
for i in range(1, 4):
    for j in range(1, 4):
        print(i * j, end="\t")
    print()          # newline after each row
_____ Output _____
1  2  3
2  4  6
3  6  9
```

```
n = 4
for i in range(1, n + 1):
    print(" " * (n - i) + "*" * (2 * i - 1))
_____ Output _____
*
***
*****
*****
```

© INTERVIEW	<code>break</code> exits only the INNERMOST loop. To leave both, use a function with <code>return</code> , a flag variable, or <code>for/else</code> .
--------------------	--

2.5 break, continue, pass

Keyword	Effect	Typical use
break	leave the loop immediately	stop searching once found
continue	skip rest of this round, go to next round	skip invalid items
pass	do nothing (placeholder)	empty function/class/branch you will write later

```
for x in [3, -1, 4, 0, 9, 7]:
    if x < 0:
        continue      # skip negatives
    if x == 0:
        break          # stop entirely at zero
    print(x, end=" ")
```

_____ Output _____

3 4

```
def todo():
    pass                # valid empty body; a bare "def todo():" would be SyntaxError
print(todo())
```

_____ Output _____

None

2.6 Loop else

A loop can have an else block. It runs **only** if the loop finished without hitting **break**. Think of it as "no-break". It is perfect for search problems.

```
for n in [3, 5, 7]:
    if n % 2 == 0:
        print("found even", n)
        break
```

```
else:
    print("no even number")    # runs: break never happened
```

_____ Output _____

no even number

```
def is_prime(n):
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            break
    else:
        return True           # no divisor found
    return False
print([n for n in range(2, 20) if is_prime(n)])
```

_____ Output _____

[2, 3, 5, 7, 11, 13, 17, 19]

© INTERVIEW

Classic quiz: `for ... else` prints the else branch when the loop is empty or ends normally. It does NOT mean "if the loop never ran". `while ... else` follows the same rule.

2.7 Iterating over strings, lists and dictionaries

Strings and lists (by value, by index, by both)

```
for ch in "hey":
    print(ch, end=" ")
print()
arr = [10, 20, 30]
for i in range(len(arr)):      # by index: only when you need i
    print(i, arr[i], end=" | ")
print()
for i, v in enumerate(arr):    # better: index AND value
    print(i, v, end=" | ")
_____ Output _____
h e y
0 10 | 1 20 | 2 30 |
0 10 | 1 20 | 2 30 |
```

Dictionaries

```
d = {"a": 1, "b": 2, "c": 3}
for k in d:                    # keys (default)
    print(k, end=" ")
print()
for v in d.values():
    print(v, end=" ")
print()
for k, v in d.items():         # key and value together
    print(k, "->", v, end=" ")
print()
print(list(d), sum(d.values()))
_____ Output _____
a b c
1 2 3
a -> 1 b -> 2 c -> 3
['a', 'b', 'c'] 6
```

⚠ COMMON MISTAKE

Never add or remove items from a list/dict while looping over it. Lists skip elements; dicts raise `RuntimeError: dictionary changed size during iteration`. Loop over a copy (for `x in a[:]`) or build a new collection.

```
a = [1, 2, 3, 4, 5, 6]
for x in a:                    # BUG: removal shifts elements
    if x % 2 == 0:
        a.remove(x)
print(a)
b = [1, 2, 3, 4, 5, 6]
b = [x for x in b if x % 2 != 0] # correct: build a new list
print(b)
_____ Output _____
[1, 3, 5]
[1, 3, 5]
```

```
d = {"a": 1, "b": 2}
try:
    for k in d:
        d["new_" + k] = 0      # size changes
except RuntimeError as e:
    print("RuntimeError:", e)
_____ Output _____
RuntimeError: dictionary changed size during iteration
```

2.8 enumerate() and zip()

`enumerate(iterable, start=0)` yields (index, value) pairs. `zip(a, b, ...)` yields tuples pairing items position by position and **stops at the shortest** iterable.

```
for i, name in enumerate(["A", "B", "C"], start=1):
    print(i, name)
names = ["A", "B", "C"]; marks = [90, 80, 70]
for n, m in zip(names, marks):
    print(n, m)
print(dict(zip(names, marks)))
```

_____ Output _____

```
1 A
2 B
3 C
A 90
B 80
C 70
{'A': 90, 'B': 80, 'C': 70}
```

```
print(list(zip("abc", [1, 2])))          # shortest wins
from itertools import zip_longest
print(list(zip_longest("abc", [1, 2], fillvalue=0)))
pairs = [(1, "x"), (2, "y")]
nums, letters = zip(*pairs)              # "unzip"
print(nums, letters)
```

_____ Output _____

```
[('a', 1), ('b', 2)]
[('a', 1), ('b', 2), ('c', 0)]
(1, 2) ('x', 'y')
```

✂ SHORTCUT

Replace `for i in range(len(arr))` with `for i, x in enumerate(arr)`. Compare two lists with `for a, b in zip(A, B)`. Neighbours: `for a, b in zip(arr, arr[1:])`.

◆ DEEP CONCEPT

`enumerate` and `zip` return lazy **iterators**: they can be consumed only once. `e = enumerate(x); list(e); list(e)` gives the full list, then `[]`.

2.9 Common loop patterns

These small patterns solve hundreds of interview problems. Learn them until you can type them without thinking.

Accumulator, counter and running max

```
nums = [4, 9, 2, 7]
total = 0
for x in nums: total += x                # accumulator
evens = 0
for x in nums:
    if x % 2 == 0: evens += 1            # counter
best = nums[0]
for x in nums:
    if x > best: best = x                 # running max
print(total, evens, best)
```

_____ Output _____

```
22 2 9
```

Frequency count with a dictionary

```
freq = {}
for ch in "banana":
    freq[ch] = freq.get(ch, 0) + 1
print(freq)
```

_____ Output _____

```
{'b': 1, 'a': 3, 'n': 2}
```

Seen-set for duplicates, and Two Sum

```
seen, dup = set(), None
for x in [3, 1, 4, 1, 5]:
    if x in seen:
        dup = x; break
    seen.add(x)
print("first duplicate:", dup)
nums, target, seen = [2, 7, 11, 15], 9, {}
for i, x in enumerate(nums):
    if target - x in seen:
        print("indices", seen[target - x], i); break
    seen[x] = i
```

_____ Output _____
first duplicate: 1
indices 0 1

Two pointers, prefix sum, sliding window

```
s = "level"; l, r = 0, len(s) - 1          # two pointers
ok = True
while l < r:
    if s[l] != s[r]: ok = False; break
    l += 1; r -= 1
print("palindrome:", ok)
arr = [2, 4, 6, 8]; prefix = [0]          # prefix sum
for x in arr: prefix.append(prefix[-1] + x)
print(prefix, "sum of arr[1..2] =", prefix[3] - prefix[1])
a, k = [1, 3, 2, 6, -1, 4], 3              # window of size k
cur = sum(a[:k]); best = cur
for right in range(k, len(a)):
    cur += a[right] - a[right - k]
    best = max(best, cur)
print("best window sum:", best)
```

_____ Output _____
palindrome: True
[0, 2, 6, 12, 20] sum of arr[1..2] = 10
best window sum: 11

Digits, circular index, building strings

```
n, digits = 4096, []
while n:
    n, d = divmod(n, 10)                  # quotient and remainder together
    digits.append(d)
print(digits[::-1])
arr = [10, 20, 30]
print([arr[(i + 1) % len(arr)] for i in range(len(arr))]) # next, wraps around
parts = []
for i in range(1, 6): parts.append(str(i))
print("-".join(parts))                   # join once; don't += in a big loop
```

_____ Output _____
[4, 0, 9, 6]
[20, 30, 10]
1-2-3-4-5

© INTERVIEW	Most "loop" interview questions are one of: accumulate, count, find max/min, search-and-break, frequency map, seen-set, two pointers, sliding window, prefix sum. Name the pattern out loud before coding.
⚠ COMMON MISTAKE	Building a string with <code>s += x</code> inside a loop can be $O(n^2)$ for huge inputs. Collect pieces in a list and <code>"".join(...)</code> . Similarly <code>list.insert(0, x)</code> and <code>list.pop(0)</code> are $O(n)$; use <code>collections.deque</code> .

Chapter 3 · Functions

A **function** is a named, reusable block of code. It takes input, does work, and (optionally) gives a result back. Functions remove repetition and let you test small pieces independently.

3.1 Defining and calling

```
def greet(name):  
    """Return a greeting for name."""      # docstring  
    return "Hello, " + name + "!"  
  
message = greet("Vk")      # call: runs the body, gets the returned value  
print(message)  
print(greet.__doc__)  
_____ Output _____  
Hello, Vk!  
Return a greeting for name.  
  
def greet ( name ):          ← header: def + function name + parameters + colon  
    return "Hello, " + name  ← body (indented); return sends a value back  
  
greet("Vk")                 ← call: "Vk" is the argument bound to name
```

★ MUST KNOW	def only creates the function. Nothing inside runs until you CALL it with parentheses. greet is the function object; greet("x") is the result of calling it.
⚠ COMMON MISTAKE	Forgetting the parentheses: print(greet) prints the function object, not the result. Forgetting to call the function at all is a silent bug.

3.2 Parameters vs arguments

Term	Meaning	Where it appears
Parameter	the variable name in the function definition	def add(a, b): → a, b
Argument	the actual value you pass when calling	add(3, 4) → 3, 4

Mnemonic: parameters are the **parking** spaces, arguments are the cars that fill them.

```
def add(a, b):          # a, b are parameters  
    return a + b  
print(add(3, 4))        # 3, 4 are arguments  
x, y = 10, 20  
print(add(x, y))        # x, y are arguments too (their values are passed)  
_____ Output _____  
7  
30
```

◆ DEEP CONCEPT	Python passes object references ("pass by assignment"). The parameter becomes another name for the same object. Mutating a list inside a function changes the caller's list; re-assigning the parameter (l = [0]) only rebinds the local name.
-----------------------	---

```
def mutate(l):  
    l.append(4)          # changes the shared object  
def rebind(l):  
    l = [0]              # rebinds local name only  
a = [1, 2, 3]  
mutate(a); print(a)  
rebind(a); print(a)  
_____ Output _____  
[1, 2, 3, 4]  
[1, 2, 3, 4]
```

3.3 return, multiple return values, return vs print

```
def square(n):  
    return n * n  
def show_square(n):  
    print(n * n)  
r1 = square(5)  
r2 = show_square(5)  
print("r1 =", r1, "| r2 =", r2)
```

Output _____
25
r1 = 25 | r2 = None

	return	print
Purpose	hands a value back to the caller	shows text on the screen
Ends the function?	yes, immediately	no
Value afterwards	the returned object	the function returns None
Can be reused in code?	yes: $y = f(x) + 1$	no: the value is lost

★ MUST KNOW

A function with no return (or a bare return) gives back None. So `x = show_square(5)` makes x equal None. This is asked in almost every Python screening.

Multiple return values are really one tuple. Python packs return `a, b` into `(a, b)`, and you unpack it on the left side.

```
def min_max(nums):  
    return min(nums), max(nums)      # a tuple  
result = min_max([4, 1, 9])  
print(result, type(result))  
lo, hi = min_max([4, 1, 9])          # unpacking  
print(lo, hi)  
q, r = divmod(17, 5)  
print(q, r)
```

Output _____
(1, 9) <class 'tuple'>
1 9
3 2

```
def first_negative(nums):  
    for x in nums:  
        if x < 0:  
            return x                # return exits the function AND the loop  
    return None  
print(first_negative([3, 5, -2, -8]), first_negative([1, 2]))
```

Output _____
-2 None

⚠ COMMON MISTAKE

Putting return inside a loop by accident: `for x in a: total += x; return total` returns after the FIRST iteration. Indent return to match the for, not inside it.

3.4 Positional, keyword and default arguments

```
def info(name, age, city="Delhi"):
    return f"{name}, {age}, {city}"
print(info("Asha", 20))           # positional + default used
print(info("Ravi", 22, "Pune"))  # positional override
print(info(age=19, name="Meera")) # keyword: order doesn't matter
print(info("Sam", city="Goa", age=21)) # mix: positional first
```

Output

```
Asha, 20, Delhi
Ravi, 22, Pune
Meera, 19, Delhi
Sam, 21, Goa
```

Kind	Written as	Rule
Positional	<code>f(1, 2)</code>	matched by position
Keyword	<code>f(a=1, b=2)</code>	matched by name, any order
Default	<code>def f(a, b=2)</code>	used when the caller omits it

★ MUST KNOW

Rules: (1) in a call, positional arguments come BEFORE keyword ones. (2) in a definition, non-default parameters come BEFORE default ones. (3) A parameter cannot receive both a positional and keyword value.

```
def f(a, b=2):
    return a + b
try:
    f(1, a=5)
except TypeError as e:
    print("TypeError:", e)
try:
    f()
except TypeError as e:
    print("TypeError:", e)
```

Output

```
TypeError: f() got multiple values for argument 'a'
TypeError: f() missing 1 required positional argument: 'a'
```

3.5 Variable-length arguments: *args and **kwargs

Syntax	Collects	Type inside function
*args	extra positional arguments	tuple
**kwargs	extra keyword arguments	dict

```
def total(*args):
    print(args, type(args))
    return sum(args)
print(total(1, 2, 3, 4))
def show(**kwargs):
    print(kwargs, type(kwargs))
    for k, v in kwargs.items():
        print(k, "=", v)
show(name="Vk", age=20)
```

Output

```
(1, 2, 3, 4) <class 'tuple'>
10
{'name': 'Vk', 'age': 20} <class 'dict'>
name = Vk
age = 20
```

The same symbols **unpack** when used in a call: * spreads an iterable into positional arguments, ** spreads a dict into keyword arguments.

```
def add3(a, b, c): return a + b + c
nums = [1, 2, 3]
d = {"a": 10, "b": 20, "c": 30}
print(add3(*nums), add3(**d))
print(*nums) # print(1, 2, 3)
merged = {"x": 1, "y": 2} # merge dicts
print(merged)
```

Output

```
6 60
1 2 3
{'x': 1, 'y': 2}
```

© INTERVIEW

Full parameter order in a definition: `def f(pos_only, /, normal, default=1, *args, kw_only, **kwargs)`. Names `args` / `kwargs` are convention; the `*` and `**` are what matter.

3.6 Positional-only and keyword-only parameters

```
def f(a, b, /, c, d, *, e, f):
    # after * : keyword-ONLY (must write e=..., f=...)
    # between / and * : positional OR keyword
    # before / : positional-ONLY (cannot use a=..., b=...)
```

```
def f(a, b, /, c, *, d):
    return a + b + c + d
print(f(1, 2, 3, d=4)) # valid
print(f(1, 2, c=3, d=4)) # c may be keyword
try: f(a=1, b=2, c=3, d=4)
except TypeError as e: print("TypeError:", e)
try: f(1, 2, 3, 4)
except TypeError as e: print("TypeError:", e)
```

Output

```
10
10
TypeError: f() got some positional-only arguments passed as keyword arguments: 'a, b'
TypeError: f() takes 3 positional arguments but 4 were given
```

◆ DEEP CONCEPT

Built-ins use these: `len(obj, /)` is positional-only; `sorted(iterable, /, *, key=None, reverse=False)` forces `key` and `reverse` to be keywords. A bare `*args` also makes every later parameter keyword-only.

✂ SHORTCUT

Keyword-only parameters make calls self-documenting: `connect(host, *, timeout=5, retries=3)` is called as `connect(h, timeout=10)`, never by mistaken position.

3.7 The mutable default argument trap

Default values are evaluated **once**, when `def` runs, not every time you call the function. A mutable default (list, dict, set) is therefore **shared between all calls**.

```
def add_item(x, bucket=[]): # BUG
    bucket.append(x)
    return bucket
print(add_item(1))
print(add_item(2)) # remembers 1 !
print(add_item.__defaults__) # the one shared list lives here
```

Output

```
[1]
[1, 2]
([1, 2],)
```

```
def add_item(x, bucket=None):    # FIX: None sentinel
    if bucket is None:
        bucket = []
    bucket.append(x)
    return bucket
print(add_item(1), add_item(2), add_item(3, [9]))
```

Output

```
[1] [2] [9, 3]
```

◎ INTERVIEW

Top-5 Python trick question. The answer has two parts: default expressions run once at definition time, and lists are mutable so the same object accumulates changes. Fix: default None, create inside. Immutable defaults (0, "", None, tuples) are safe.

3.8 Lambda functions

A **lambda** is a tiny anonymous function limited to a single expression. It returns that expression's value automatically (no return keyword).

```
def square(x):          square = lambda x: x * x
    return x * x          ↑      ↑      ↑
                        keyword params expression (value returned)
```

```
add = lambda a, b: a + b
print(add(2, 3), (lambda x: x * 2)(21))    # can be called immediately
words = ["pear", "fig", "banana"]
print(sorted(words, key=len))
pairs = [(1, 5), (2, 2), (3, 8)]
print(sorted(pairs, key=lambda p: p[1], reverse=True))
print(max(pairs, key=lambda p: p[1]))
print(list(map(lambda x: x * x, [1, 2, 3])))
print(list(filter(lambda x: x % 2 == 0, range(10))))
```

Output

```
5 42
['fig', 'pear', 'banana']
[(3, 8), (1, 5), (2, 2)]
(3, 8)
[1, 4, 9]
[0, 2, 4, 6, 8]
```

⚠ COMMON MISTAKE

Late binding: lambdas look up outer variables when CALLED, not when created. `[lambda: i for i in range(3)]` all return 2. Freeze the value with a default: `lambda i=i: i`.

```
fs = [lambda: i for i in range(3)]
print([f() for f in fs])
fs = [lambda i=i: i for i in range(3)]
print([f() for f in fs])
```

Output

```
[2, 2, 2]
[0, 1, 2]
```

★ MUST KNOW

Use lambda for short throw-away functions passed as key=. If it needs a name, statements, or more than one line, write a normal def. PEP 8 discourages `f = lambda ....`

⚡ SHORTCUT

Prefer a comprehension over map/filter with lambda: `[x*x for x in a]` and `[x for x in a if x%2==0]` are usually clearer.

3.9 First-class functions and functions as objects

In Python a function is an **object** like any other. It can be assigned to a variable, stored in a list or dict, passed as an argument, and returned from another function. This is what "first-class" means. A function that takes or returns functions is a **higher-order function**.

```
def hello(): return "hi"
greet = hello          # another name for the SAME object
print(greet(), greet is hello)
print(type(hello), hello.__name__, callable(hello))
hello.calls = 0        # objects can carry attributes
print(hello.calls)

_____ Output _____

hi True
<class 'function'> hello True
0
```

```
def apply(func, value):      # function as argument
    return func(value)
print(apply(len, "python"), apply(str.upper, "abc"), apply(lambda x: x + 1, 9))
def multiplier(n):          # function returned from a function
    def times(x):
        return x * n        # closure: remembers n
    return times
double = multiplier(2)
print(double(8), multiplier(5)(3))

_____ Output _____

6 ABC 10
16 15
```

```
ops = {"+": lambda a, b: a + b, "-": lambda a, b: a - b,
      "*": lambda a, b: a * b}      # dispatch table: dict of functions
print(ops["*"](6, 7))

_____ Output _____

42
```

◆ DEEP CONCEPT

A **closure** is an inner function that remembers variables of the enclosing function even after that function has finished. Decorators (@decorator) are just higher-order functions: @d def f means f = d(f). You will study them in a later part.

◎ INTERVIEW

"Are functions objects in Python?" Yes. They have a type (function), attributes (__name__, __doc__, __defaults__) and can be passed around. A dict of functions replaces long if/elif chains.

3.10 Recursion

Recursion means a function calls itself on a smaller version of the problem. Every recursive function needs: a **base case** (stops the recursion) and a **recursive case** (moves toward the base case).

```
def factorial(n):
    if n == 0:                # base case
        return 1
    return n * factorial(n - 1) # recursive case
print(factorial(5))

_____ Output _____

120
```

```
factorial(3)
└─ 3 * factorial(2)
   └─ 2 * factorial(1)
      └─ 1 * factorial(0)
         └─ returns 1 (base)
            returns 1 * 1 = 1
               returns 2 * 1 = 2
                  returns 3 * 2 = 6
```

Call stack (grows, then unwinds)

factorial(0)	← returns 1
factorial(1)	← returns 1*1 = 1
factorial(2)	← returns 2*1 = 2
factorial(3)	← returns 3*2 = 6

⚠ COMMON MISTAKE	Missing or unreachable base case means infinite recursion. Python stops it with <code>RecursionError</code> (default limit ≈ 1000 frames).
-------------------------	--

```
import sys
print(sys.getrecursionlimit())
def forever(n): return forever(n + 1)
try: forever(0)
except RecursionError as e: print("RecursionError:", e)
```

Output

```
1000
RecursionError: maximum recursion depth exceeded
```

Naive Fibonacci recomputes the same values again and again: about $O(2^n)$. **Memoization** stores results and makes it $O(n)$.

```
from functools import lru_cache
calls = 0
def fib_slow(n):
    global calls; calls += 1
    return n if n < 2 else fib_slow(n - 1) + fib_slow(n - 2)
print(fib_slow(20), "calls:", calls)
@lru_cache(maxsize=None)
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
print(fib(80))
```

Output

```
6765 calls: 21891
23416728348467685
```

	Recursion	Iteration (loop)
Idea	function calls itself on a smaller input	repeat with for / while
Memory	one stack frame per call: $O(\text{depth})$	usually $O(1)$ extra
Speed	function-call overhead	faster in Python
Limits	recursion limit (~ 1000); no tail-call optimisation	none
Best for	trees, graphs (DFS), backtracking, divide and conquer, nested data	simple counting, sums, scanning arrays
Readability	natural for self-similar problems	natural for linear sequences

© INTERVIEW	Python does NOT do tail-call optimisation, so deep recursion (e.g. 100 000 levels) fails. Convert to a loop or an explicit stack. Every recursion can be rewritten as iteration (possibly with a stack).
--------------------	--

3.11 Scope: local, global, LEGB

Scope is the region where a name is visible. A variable created inside a function is **local** and disappears when the function ends. A variable created at the top level of a file is **global**.

```
x = "global"
def f():
    x = "local"          # a NEW local variable, hides the global one
    print("inside:", x)
f()
print("outside:", x)
```

Output

```
inside: local
outside: global
```

Python looks up a name in this order and stops at the first match:

```

B Built-in: len, print, range, Exception ...
  G Global: module level of this file
    E Enclosing: outer function of a nested def
      L Local: inside the current function

```

Search order: **L → E → G → B**

```

x = "global-x"
def outer():
    x = "enclosing-x"
    def inner():
        x = "local-x"
        print(x)                # L
    def inner2():
        print(x)                # E (no local x)
    inner(); inner2()
outer()
def uses_global(): print(x)      # G
uses_global()
print(len("abc"))               # B

```

_____ Output _____

```

local-x
enclosing-x
global-x
3

```

Reading vs assigning: the UnboundLocalError trap

Reading a global inside a function works automatically. But **assigning** to a name anywhere in a function makes that name local for the WHOLE function.

```

count = 0
def bad():
    count += 1          # assignment => count is local, but has no value yet
    return count
try: bad()
except UnboundLocalError as e: print("UnboundLocalError:", e)

```

_____ Output _____

```

UnboundLocalError: cannot access local variable 'count' where it is not associated
with a value

```

global and nonlocal

```

count = 0
def inc():
    global count        # "use the module-level count"
    count += 1
inc(); inc()
print(count)

```

_____ Output _____

```

2

```

```
def make_counter():
    n = 0
    def step():
        nonlocal n      # "use the enclosing function's n"
        n += 1
        return n
    return step
c = make_counter()
print(c(), c(), c())
c2 = make_counter()    # separate closure, separate n
print(c2())
```

_____ Output _____

```
1 2 3
1
```

Keyword	Refers to	Use
global x	variable in module (file) scope	rebind a global from inside a function
nonlocal x	variable in the nearest enclosing function (not global)	rebind an outer function's variable from an inner one

⚠ COMMON MISTAKE	You do NOT need global to mutate a global list/dict: <code>items.append(1)</code> works because you are not rebinding the name. You DO need it for <code>items = []</code> or <code>count += 1</code> .
-------------------------	---

🔪 SHORTCUT	Avoid global in real code. Pass values in as arguments and return results. Use nonlocal mainly for closures, or use a small class.
-------------------	--

💎 DEEP CONCEPT	Function parameters are locals. Loop variables and variables assigned in an if block leak into the enclosing scope (Python has function scope, not block scope). Comprehension variables do NOT leak in Python 3.
-----------------------	---

```
for i in range(3):
    j = i * 2
print(i, j)                # both still exist after the loop
squares = [k * k for k in range(3)]
try: print(k)
except NameError as e: print("NameError:", e)
```

_____ Output _____

```
2 4
NameError: name 'k' is not defined
```

Chapter 4 · Concept Clinic: Side-by-Side

Interviewers love "what is the difference between X and Y". Each pair below is short enough to recite.

4.1 Parameter vs argument

	Parameter	Argument
Where	in def line	in the call
Nature	a name (placeholder)	a value / object
Example	<code>def f(a): → a</code>	<code>f(10) → 10</code>

4.2 return vs print

print **displays**, return **delivers**. Only a returned value can be stored, tested, or chained. A function that only prints returns None.

4.3 Local vs global

	Local	Global
Created	inside a function	top level of the module
Lifetime	one call	whole program run
Visible	only inside that function	everywhere (readable); rebinding needs global
Best practice	preferred	keep to constants (<code>MAX_SIZE = 100</code>)

4.3b Recursion vs iteration

Both repeat work. Recursion uses the call stack and is elegant for tree/graph/backtracking problems. Iteration uses loops, is faster and has no depth limit in Python. See the table in section 3.10.

4.4 Lambda vs def

	lambda	def
Name	anonymous (can be assigned)	named
Body	one expression only	any number of statements
return	implicit	explicit return
Docstring / annotations	no	yes
Use	short key= functions	everything else

4.5 First-class functions and functions as objects

Because functions are objects you can: store them (`ops = {"+": add}`), pass them (`sorted(a, key=f)`), return them (closures), give them attributes, and inspect them (`f.__name__`, `f.__defaults__`). That is why decorators, callbacks and dispatch tables are possible.

4.6 Mutable default argument trap (one-line summary)

Defaults are created once at def time. Never use `[]` or `{}` as a default. Use `None` and build a fresh object inside.

4.7 == vs is, and args vs kwargs

Pair	Difference
<code>==</code> vs <code>is</code>	<code>==</code> compares values; <code>is</code> compares object identity (same object in memory). Use <code>is</code> only with <code>None</code> , <code>True</code> , <code>False</code> .
<code>*args</code> vs <code>**kwargs</code>	<code>*args</code> gathers extra positional args into a tuple; <code>**kwargs</code> gathers extra keyword args into a dict.
<code>break</code> vs <code>continue</code>	<code>break</code> leaves the loop; <code>continue</code> jumps to the next round.
<code>for</code> vs <code>while</code>	<code>for</code> iterates over a known iterable; <code>while</code> repeats until a condition is false.
<code>global</code> vs <code>nonlocal</code>	<code>global</code> targets module scope; <code>nonlocal</code> targets the nearest enclosing function.
<code>range</code> vs <code>list(range)</code>	<code>range</code> is a lazy $O(1)$ -memory sequence; <code>list(range)</code> materialises every number.

Chapter 5 · Real Coding Examples

These mini-programs combine conditions, loops and functions the way real code (and interviews) do.

5.1 FizzBuzz

```
def fizzbuzz(n):
    out = []
    for i in range(1, n + 1):
        if i % 15 == 0: out.append("FizzBuzz")
        elif i % 3 == 0: out.append("Fizz")
        elif i % 5 == 0: out.append("Buzz")
        else: out.append(str(i))
    return out
print(fizzbuzz(15))
```

Output

```
['1', '2', 'Fizz', '4', 'Buzz', 'Fizz', '7', '8', 'Fizz', 'Buzz', '11', 'Fizz', '13', '14', 'FizzBuzz']
```

© INTERVIEW

Check % 15 (or both 3 and 5) FIRST. Checking 3 first would print "Fizz" for 15.

5.2 Prime check and GCD

```
def gcd(a, b):
    while b:
        a, b = b, a % b      # Euclid's algorithm
    return abs(a)
print(gcd(48, 18), gcd(17, 5))
```

Output

```
6 1
```

5.3 Palindrome and vowel count

```
def is_palindrome(s):
    clean = [c.lower() for c in s if c.isalnum()]
    return clean == clean[::-1]
print(is_palindrome("A man, a plan, a canal: Panama"), is_palindrome("hello"))
print(sum(1 for c in "Education".lower() if c in "aeiou"))
```

Output

```
True False
5
```

5.4 Binary search (function + while loop)

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target: return mid
        elif arr[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1
a = [1, 3, 5, 7, 9, 11]
print(binary_search(a, 7), binary_search(a, 4))
```

Output

```
3 -1
```

5.5 Calculator with a dispatch table

```
def calc(op, a, b):
    ops = {"+": lambda x, y: x + y, "-": lambda x, y: x - y,
           "*": lambda x, y: x * y,
           "/": lambda x, y: x / y if y != 0 else "cannot divide by zero"}
    return ops.get(op, lambda x, y: "unknown operator")(a, b)
print(calc("+", 6, 3), calc("/", 6, 0), calc("%", 1, 1))
```

Output

```
9 cannot divide by zero unknown operator
```

5.6 Flatten a nested list (recursion)

```
def flatten(items):
    flat = []
    for it in items:
        if isinstance(it, list):
            flat.extend(flatten(it))      # recursive case
        else:
            flat.append(it)              # base case
    return flat
print(flatten([1, [2, [3, 4]], 5, [[6]]]))
```

Output

```
[1, 2, 3, 4, 5, 6]
```

5.7 Password validator (conditions + loops + any/all)

```
def check_password(p):
    problems = []
    if len(p) < 8: problems.append("too short")
    if not any(c.isdigit() for c in p): problems.append("needs a digit")
    if not any(c.isupper() for c in p): problems.append("needs an uppercase letter")
    return "OK" if not problems else ", ".join(problems)
print(check_password("abc"), "|", check_password("Strong123"))
```

Output

```
too short, needs a digit, needs an uppercase letter | OK
```

5.8 Function with *args, **kwargs and keyword-only settings

```
def report(title, *scores, curve=0, **meta):
    avg = sum(scores) / len(scores) + curve if scores else 0
    extras = ", ".join(f"{k}={v}" for k, v in sorted(meta.items()))
    return f"{title}: avg={avg:.1f}" + (f" ({extras})" if extras else "")
print(report("Test", 80, 90, 70, curve=5, year=2026, grade="X"))
```

Output

```
Test: avg=85.0 (grade=X, year=2026)
```

Chapter 6 · Revision Notes

Conditions in one page

- Chain runs the FIRST true branch only. Separate ifs are independent. Order elif from most specific to most general.
- Ternary: a if cond else b. In comprehensions: transform → before for; filter → after for.
- Falsy: False None 0 0.0 "" [] {} set() range(0). Everything else truthy ("0", "False", [0] too).
- Prefer if arr: over len(arr) > 0; use is None / is not None for None; use x in (1, 2) not x == 1 or 2.
- Chained comparison: $1 < x < 10 = 1 < x \text{ and } x < 10$.
- and/or short-circuit and return an operand, not always a bool. Precedence: not > and > or.
- Floats: compare with math.isclose. is = identity, == = equality.

Loops in one page

- for for iterables and known counts; while when the number of rounds depends on a condition.
- range(start, stop, step): stop excluded; lazy; O(1) memory; step -1 to count down.
- break exits the innermost loop, continue skips to the next round, pass does nothing.
- Loop else runs only if the loop finished WITHOUT break.
- Strings: chars. Lists: items. Dicts: keys (use .values(), .items()). enumerate(x, start=0), zip stops at the shortest.
- Never add/remove items while iterating; iterate a copy or build a new collection.
- The loop variable survives after the loop. Nested loops multiply cost ($O(n^2)$).
- Patterns: accumulator, counter, running max, frequency dict, seen-set, two pointers, sliding window, prefix sum, digit loop with divmod.

Functions in one page

- Parameters are in def, arguments are in the call. Python passes object references.
- No return ⇒ None. return a, b ⇒ a tuple. return ends the function at once. print shows, return delivers.
- Order in def: positional-only /, normal, defaults, *args, keyword-only, **kwargs. In calls: positional before keyword.
- *args → tuple, **kwargs → dict. In calls * and ** unpack.
- Defaults are evaluated once at def time: NEVER use [] / {} as default; use None.
- lambda args: expression. Single expression only. Watch late binding in loops.
- Functions are first-class objects: assign, pass, return, store, give attributes. Closures remember outer variables.
- Recursion needs a base case; depth limit ~1000; use memoization (lru_cache) for overlapping subproblems.
- LEGB lookup: Local → Enclosing → Global → Built-in. Assigning in a function makes the name local. global rebinds module names; nonlocal rebinds enclosing names.

Chapter 7 · Function Cheat Sheet

Task	Syntax	Notes
Define	<code>def f(a, b): ...</code>	body indented; docstring first line
Call	<code>f(1, 2)</code>	parentheses required
Return value	<code>return x</code>	exits function; default is None
Return several	<code>return a, b</code>	tuple; unpack: <code>x, y = f()</code>
Default arg	<code>def f(a, b=10)</code>	evaluated once; use None for mutables
Keyword call	<code>f(b=2, a=1)</code>	order free; positional first
Var. positional	<code>def f(*args)</code>	<code>args</code> is a tuple
Var. keyword	<code>def f(**kw)</code>	<code>kw</code> is a dict
Positional-only	<code>def f(a, /, b)</code>	<code>a</code> cannot be passed by name
Keyword-only	<code>def f(a, *, b)</code>	<code>b</code> must be passed by name
Unpack call	<code>f(*lst) f(**dct)</code>	spreads items / key-values
Lambda	<code>lambda x: x + 1</code>	one expression; used in <code>key=</code>
Nested / closure	<code>def outer(): def inner(): ...</code>	inner sees outer's variables
Global write	<code>global x</code>	rebind module variable
Enclosing write	<code>nonlocal x</code>	rebind outer function variable
Recursion	<code>def f(n): if n<=0: return 0; return n+f(n-1)</code>	base case first
Memoize	<code>@functools.lru_cache(None)</code>	caches results by arguments
Function as value	<code>g = f apply(f, v) {"k": f}</code>	functions are objects
Inspect	<code>f.__name__ f.__doc__ f.__defaults__ callable(f)</code>	introspection
Type hints	<code>def f(a: int, b: str = "x") -> bool:</code>	not enforced at runtime

Argument-order diagram

```
def f( p1, p2, /, a, b=0, *args, k1, k2=5, **kwargs ):
    positional-    normal (pos    extra    keyword-    extra
      only          or keyword)  positional only    keyword

f(1, 2, 3, 4, 5, 6, k1=7, z=9)
  p1=1 p2=2  a=3 b=4  args=(5, 6)  k1=7 k2=5  kwargs={'z': 9}
```

Decision guide

Situation	Use
Repeat a known number of times / over items	for
Repeat until something changes	while
Need index and value	enumerate
Walk two lists together	zip
Search and know if nothing found	loop else or early return
Choose between 2 values in one line	ternary
Many values compared to one variable	if/elif, match, or dict dispatch
Tiny function for sorting key	lambda
Self-similar problem (tree, backtracking)	recursion (+ memoize)
Function needs its own private state	closure with nonlocal (or a class)

Chapter 8 · Practice Questions

Try every question on paper or in an editor **before** opening the solutions in Chapter 9. Predict outputs first, then run them.

Part A · 30 Conceptual Questions

- C1.** What is the difference between `=` and `==`?
- C2.** When is an `elif` evaluated?
- C3.** What does `x or y` return? And `x and y`?
- C4.** List the falsy values in Python.
- C5.** What is short-circuit evaluation and why is it useful?
- C6.** What does `1 < x < 10` mean and how is it different from other languages?
- C7.** Difference between `is` and `==`? Why write `if x is None`?
- C8.** Write the syntax of the ternary operator and explain the order.
- C9.** Is `range(5)` a list? What are its properties?
- C10.** Differentiate `break`, `continue` and `pass`.
- C11.** When does the `else` of a `for` loop run?
- C12.** Why should you not remove items from a list while iterating over it?
- C13.** What does `enumerate()` return, and what is `start=`?
- C14.** What happens in `zip(a, b)` if the lengths differ?
- C15.** When would you choose `while` over `for`?
- C16.** Define parameter and argument.
- C17.** Explain `return` vs `print`.
- C18.** What does a function return if it has no `return` statement?
- C19.** How do multiple return values work?
- C20.** State the rules for the order of parameters and arguments.
- C21.** What are `*args` and `**kwargs`? What types are they inside the function?
- C22.** What are positional-only and keyword-only parameters?
- C23.** Explain the mutable default argument bug and its fix.
- C24.** What is a `lambda`? What are its limitations?
- C25.** What does "functions are first-class objects" mean?
- C26.** Explain the LEGB rule.
- C27.** Difference between `global` and `nonlocal`?
- C28.** Why does `count += 1` inside a function raise `UnboundLocalError`?
- C29.** What are the essential parts of a recursive function and what if one is missing?
- C30.** Compare recursion and iteration. Does Python optimise tail calls?

Part B · 30 Output Questions

What is printed? If the code raises an error, name the error.

01.

```
print(1 < 2 < 3, 3 > 2 > 2)
```

02.

```
print(0 or "" or "hi", 1 and 2 and 0)
```

03.

```
print(bool("False"), bool(""), bool([0]), bool(0.0))
```

04.

```
print(True or False and False, (True or False) and False)
```

05.

```
x = 10
if x > 5:
    print("A")
elif x > 8:
    print("B")
else:
    print("C")
```

06.

```
for i in range(3):
    pass
print(i)
```

07.

```
for i in range(5, 0, -2):
    print(i, end=" ")
```

08.

```
print(list(range(10, 0, -3)), list(range(5, 1)))
```

09.

```
for n in [1, 3, 5]:
    if n % 2 == 0:
        break
else:
    print("no even")
```

010.

```
i = 0
while i < 3:
    i += 1
    if i == 2:
        continue
    print(i)
```

011.

```
for i in range(3):
    for j in range(3):
        if j == 1:
            break
        print(i, j)
```

O12.

```
a = [1, 2, 3, 4]
for x in a:
    if x % 2 == 0:
        a.remove(x)
print(a)
```

O13.

```
print(list(zip("abc", [1, 2])))
```

O14.

```
print(list(enumerate("ab", start=1)))
```

O15.

```
d = {"a": 1, "b": 2}
for k in d:
    d[k] *= 10
print(d)
```

O16.

```
def f():
    print("hi")
x = f()
print(x)
```

O17.

```
def f(a, b=2, *args, **kw):
    print(a, b, args, kw)
f(1, 2, 3, 4, x=5)
```

O18.

```
def add(x, lst=[]):
    lst.append(x)
    return lst
print(add(1), add(2), add(3, []))
```

O19.

```
def f():
    return 1, 2
print(f(), type(f()))
```

O20.

```
x = 10
def f():
    x = 20
f()
print(x)
```

O21.

```
x = 1
def f():
    x += 1
    return x
print(f())
```

O22.

```
def outer():
    x = 1
    def inner():
        nonlocal x
        x += 1
    inner()
    return x
print(outer())
```

O23.

```
fs = [lambda: i for i in range(3)]
print([f() for f in fs])
```

O24.

```
fs = [lambda i=i: i for i in range(3)]
print([f() for f in fs])
```

O25.

```
def f(n):
    if n == 0:
        return 0
    return n + f(n - 1)
print(f(4))
```

O26.

```
print(sorted(["bb", "a", "ccc"], key=lambda s: -len(s)))
```

O27.

```
def f(x, /, y, *, z):
    return x + y + z
print(f(1, 2, z=3))
print(f(1, 2, 3))
```

O28.

```
def f(l):
    l.append(4)
    l = [0]
x = [1, 2, 3]
f(x)
print(x)
```

O29.

```
print(type(lambda: 0), (lambda x: x * 2)(4))
```

O30.

```
for i in range(3):
    pass
else:
    print("done", i)
```

Part C · 30 Coding Questions

- K1.** Write `classify(n)` returning "zero", "even" or "odd".
- K2.** Write `largest3(a, b, c)` without using `max`.
- K3.** Write `is_leap(year)`.
- K4.** Write `fizzbuzz(n)` returning a list for 1..n.
- K5.** Write `grade(marks)`: $A \geq 90$, $B \geq 75$, $C \geq 50$, else F. Raise `ValueError` outside 0..100.
- K6.** Write `digit_sum(n)` using a while loop.
- K7.** Write `reverse_int(n)` that reverses the digits of an integer (keep the sign).
- K8.** Write `count_vowels(s)` (case-insensitive).
- K9.** Write `is_palindrome(s)` ignoring case, spaces and punctuation.
- K10.** Write `is_prime(n)` (loop only up to the square root).
- K11.** Write `primes_upto(n)` using the Sieve of Eratosthenes.
- K12.** Write `gcd(a, b)` with Euclid's algorithm (iterative).
- K13.** Write a recursive `factorial(n)` that raises `ValueError` for negative n.
- K14.** Write `fib(n)` iteratively (`fib(0)=0`, `fib(1)=1`).
- K15.** Write a recursive `fib_memo(n)` with a memo dictionary, avoiding the mutable-default trap.
- K16.** Write `char_freq(s)` returning a dict of character counts.
- K17.** Write `first_unique(s)` returning the first non-repeating character or `None`.
- K18.** Write `two_sum(nums, target)` returning the two indices, in one pass.
- K19.** Write `second_largest(nums)` (distinct values, no sorting). Return `None` if it does not exist.
- K20.** Write `dedupe(items)` removing duplicates while keeping order.
- K21.** Write recursive `flatten(nested)` for arbitrarily nested lists.
- K22.** Write `power(x, n)` recursively with fast exponentiation ($n \geq 0$).
- K23.** Write `binary_search(arr, target)` returning the index or -1.
- K24.** Write `average(*nums)` returning 0 when nothing is passed.
- K25.** Write `describe(**kwargs)` returning "a=1, b=2" with keys sorted.
- K26.** Fix the mutable default bug: write `append_to(x, lst=None)`.
- K27.** Write `make_counter()` returning a function that gives 1, 2, 3, ... on successive calls (closure).
- K28.** Sort a list of pairs by second element, ties broken by first element, using a lambda:
`sort_pairs(pairs)`.
- K29.** Write `triangle(n)` returning a right-angled star triangle as one string.
- K30.** Write `rotate_right(lst, k)` rotating a list right by k (k may exceed the length).

Chapter 9 · Complete Solutions

Part A · Conceptual Answers

C1. What is the difference between = and ==?

= assigns a value to a name. == compares two values and gives True/False. `if x = 5:` is a `SyntaxError`.

C2. When is an `elif` evaluated?

Only if every earlier `if/elif` condition in the same chain was false. Once one branch runs, the rest are skipped.

C3. What does `x or y` return? And `x and y`?

`x or y` returns `x` if `x` is truthy, else `y`. `x and y` returns `x` if `x` is falsy, else `y`. They return operands, not necessarily booleans.

C4. List the falsy values in Python.

`False`, `None`, `0`, `0.0`, `0j`, `""`, `[]`, `()`, `{}`, `set()`, `range(0)`, and objects whose `__bool__` returns `False` or `__len__` returns `0`.

C5. What is short-circuit evaluation and why is it useful?

The right operand of `and/or` is evaluated only if needed. It lets you guard errors: `b != 0 and a / b > 1, lst and lst[0]`.

C6. What does `1 < x < 10` mean and how is it different from other languages?

It means `1 < x` and `x < 10` with `x` evaluated once. In C-like languages `1 < x < 10` compares a boolean with `10`, which is wrong.

C7. Difference between `is` and `==`? Why write `if x is None`?

`==` compares values, `is` compares identity. `None` is a singleton, so `is` is exact, faster, and cannot be fooled by a custom `__eq__`.

C8. Write the syntax of the ternary operator and explain the order.

`value_if_true if condition else value_if_false`. The condition is evaluated first; then only the chosen value.

C9. Is `range(5)` a list? What are its properties?

No. It is a lazy immutable sequence storing start/stop/step. `O(1)` memory, supports `len`, indexing, slicing, and fast `in`; use `list(range(5))` to materialise.

C10. Differentiate `break`, `continue` and `pass`.

`break` exits the loop, `continue` skips to the next iteration, `pass` is a no-op placeholder needed where a statement is required.

C11. When does the `else` of a `for` loop run?

When the loop ends without `break` (including when the iterable is empty). It does not run if `break` executed.

C12. Why should you not remove items from a list while iterating over it?

Removal shifts later items left, so the hidden index skips elements. Iterate over a copy or build a new list with a comprehension.

C13. What does `enumerate()` return, and what is `start=`?

A lazy iterator of `(index, value)` tuples. `start` sets the first index (default `0`), e.g. `enumerate(a, start=1)`.

C14. What happens in `zip(a, b)` if the lengths differ?

It stops at the shortest iterable, silently dropping extras. Use `itertools.zip_longest` to keep them, or `zip(..., strict=True)` (3.10+) to raise an error.

C15. When would you choose `while` over `for`?

When the number of iterations is unknown in advance and depends on a condition: digit processing, two pointers, binary search, reading until a sentinel, simulations.

C16. Define parameter and argument.

A parameter is a variable name in the function definition; an argument is the actual value supplied at the call.

C17. Explain `return` vs `print`.

`print` outputs text to the screen and returns `None`. `return` sends a value back to the caller, ends the function, and lets the value be reused.

C18. What does a function return if it has no `return` statement?

`None`.

C19. How do multiple return values work?

`return a, b` builds one tuple `(a, b)`. The caller can unpack it: `x, y = f()`.

C20. State the rules for the order of parameters and arguments.

In a definition: non-default before default; `*args` after them; keyword-only after `*`; `**kwargs` last. In a call: positional arguments before keyword arguments; no parameter may get two values.

C21. What are `*args` and `**kwargs`? What types are they inside the function?

`*args` collects extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dict. In a call, `*` and `**` unpack iterables and dicts.

C22. What are positional-only and keyword-only parameters?

Parameters before `/` must be passed by position; parameters after `*` (or after `*args`) must be passed by keyword.

C23. Explain the mutable default argument bug and its fix.

The default object is created once when `def` executes and reused across calls, so mutations persist. Use `param=None` and create the list/dict inside the function.

C24. What is a lambda? What are its limitations?

An anonymous function defined with `lambda args: expression`. Only one expression, no statements or annotations, hard to debug. Use `def` for anything larger.

C25. What does "functions are first-class objects" mean?

Functions can be assigned to variables, stored in data structures, passed as arguments, returned from functions, and have attributes, just like ints or lists.

C26. Explain the LEGB rule.

Name lookup goes Local → Enclosing (outer functions) → Global (module) → Built-in and stops at the first match.

C27. Difference between `global` and `nonlocal`?

`global` declares that a name refers to the module-level variable; `nonlocal` refers to the nearest enclosing function's variable (never the global one).

C28. Why does `count += 1` inside a function raise `UnboundLocalError`?

Assignment anywhere in the function makes `count` local for the whole function, so reading it before assignment fails. Declare `global count` or pass/return values.

C29. What are the essential parts of a recursive function and what if one is missing?

A base case and a recursive case that moves toward it. Without a base case the calls never stop and Python raises `RecursionError` (limit ≈ 1000).

C30. Compare recursion and iteration. Does Python optimise tail calls?

Recursion uses stack frames ($O(\text{depth})$ memory) and suits trees/backtracking; loops are faster and unlimited in depth. Python has NO tail-call optimisation, so convert deep recursion to loops or use memoization.

Part B · Output Answers

01. Output:

True False

Why: $3 > 2 > 2$ means $3 > 2$ and $2 > 2$; the second part is False.

02. Output:

hi 0

Why: or returns the first truthy operand ("hi"); and returns the first falsy one (0).

03. Output:

True False True False

Why: A non-empty string and a non-empty list are truthy no matter what they contain.

04. Output:

True False

Why: and binds tighter than or: first is True or (False and False).

05. Output:

A

Why: The first true branch wins; elif $x > 8$ is never checked.

06. Output:

2

Why: The loop variable survives the loop and holds the last value.

07. Output:

5 3 1

Why: Start 5, step -2 , stop 0 excluded: 5, 3, 1.

08. Output:

[10, 7, 4, 1] []

Why: Second range has the wrong direction (default step $+1$), so it is empty.

09. Output:

no even

Why: No break happened, so the loop else runs.

010. Output:

1

3

Why: continue skips the print when i is 2 (the increment was already done, so no infinite loop).

011. Output:

0 0

1 0

2 0

Why: break leaves only the inner loop, so each i prints only $j = 0$.

012. Output:

[1, 3]

Why: Removing shifts the list while iterating, so the loop skips items; the result is not what a naive reading suggests.

013. Output:

[('a', 1), ('b', 2)]

Why: zip stops at the shortest input; "c" is dropped.

014. Output:

[(1, 'a'), (2, 'b')]

Why: Pairs of (index, value), counting from 1.

O15. Output:

```
{'a': 10, 'b': 20}
```

Why: Changing values is fine; changing the SIZE during iteration would raise RuntimeError.

O16. Output:

```
hi
None
```

Why: f prints "hi" but returns None.

O17. Output:

```
1 2 (3, 4) {'x': 5}
```

Why: a=1, b=2, extra positionals go to the tuple args, x=5 goes to the dict kw.

O18. Output:

```
[1, 2] [1, 2] [3]
```

Why: First two calls share one default list, and print shows its final state twice; the third call gets a fresh list.

O19. Output:

```
(1, 2) <class 'tuple'>
```

Why: Multiple return values are packed into a tuple.

O20. Output:

```
10
```

Why: x = 20 creates a local variable; the global x is untouched.

O21. Output:

```
UnboundLocalError: cannot access local variable 'x' where it is not associated with a value
```

Why: Assignment makes x local, so x += 1 reads an unassigned local: UnboundLocalError.

O22. Output:

```
2
```

Why: nonlocal lets inner rebind the enclosing x.

O23. Output:

```
[2, 2, 2]
```

Why: Late binding: every lambda reads the same i, which is 2 after the loop.

O24. Output:

```
[0, 1, 2]
```

Why: The default argument freezes i at creation time.

O25. Output:

```
10
```

Why: 4 + 3 + 2 + 1 + 0 = 10.

O26. Output:

```
['ccc', 'bb', 'a']
```

Why: Sorted by negative length, so longest first.

O27. Output:

```
6
```

```
TypeError: f() takes 2 positional arguments but 3 were given
```

Why: Valid call prints 6. The second call passes z positionally, which keyword-only forbids: TypeError.

O28. Output:

```
[1, 2, 3, 4]
```

Why: append mutates the shared list; l = [0] only rebinds the local name.

O29. Output:

```
<class 'function'> 8
```

Why: A lambda is an ordinary function object; it can be called immediately.

O30. Output:

done 2

Why: No break, so else runs, and i is still 2.

Part C - Coding Solutions

Every solution below was executed against tests when this PDF was built.

K1. Write classify(n) returning "zero", "even" or "odd".

```
def classify(n):
    if n == 0:
        return "zero"
    return "even" if n % 2 == 0 else "odd"
```

K2. Write largest3(a, b, c) without using max.

```
def largest3(a, b, c):
    if a >= b and a >= c:
        return a
    return b if b >= c else c
```

K3. Write is_leap(year).

```
def is_leap(year):
    return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)
```

K4. Write fizzbuzz(n) returning a list for 1..n.

```
def fizzbuzz(n):
    out = []
    for i in range(1, n + 1):
        if i % 15 == 0:
            out.append("FizzBuzz")
        elif i % 3 == 0:
            out.append("Fizz")
        elif i % 5 == 0:
            out.append("Buzz")
        else:
            out.append(str(i))
    return out
```

K5. Write grade(marks): A ≥ 90, B ≥ 75, C ≥ 50, else F. Raise ValueError outside 0..100.

```
def grade(marks):
    if not 0 <= marks <= 100:
        raise ValueError("marks must be 0..100")
    if marks >= 90: return "A"
    elif marks >= 75: return "B"
    elif marks >= 50: return "C"
    return "F"
```

K6. Write digit_sum(n) using a while loop.

```
def digit_sum(n):
    n = abs(n)
    total = 0
    while n:
        n, d = divmod(n, 10)
        total += d
    return total
```

K7. Write `reverse_int(n)` that reverses the digits of an integer (keep the sign).

```
def reverse_int(n):
    sign = -1 if n < 0 else 1
    n, rev = abs(n), 0
    while n:
        n, d = divmod(n, 10)
        rev = rev * 10 + d
    return sign * rev
```

K8. Write `count_vowels(s)` (case-insensitive).

```
def count_vowels(s):
    return sum(1 for ch in s.lower() if ch in "aeiou")
```

K9. Write `is_palindrome(s)` ignoring case, spaces and punctuation.

```
def is_palindrome(s):
    clean = [c.lower() for c in s if c.isalnum()]
    return clean == clean[::-1]
```

K10. Write `is_prime(n)` (loop only up to the square root).

```
def is_prime(n):
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            return False
    return True
```

K11. Write `primes_upto(n)` using the Sieve of Eratosthenes.

```
def primes_upto(n):
    if n < 2:
        return []
    sieve = [True] * (n + 1)
    sieve[0] = sieve[1] = False
    for i in range(2, int(n ** 0.5) + 1):
        if sieve[i]:
            for j in range(i * i, n + 1, i):
                sieve[j] = False
    return [i for i, ok in enumerate(sieve) if ok]
```

K12. Write `gcd(a, b)` with Euclid's algorithm (iterative).

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return abs(a)
```

K13. Write a recursive `factorial(n)` that raises `ValueError` for negative `n`.

```
def factorial(n):
    if n < 0:
        raise ValueError("negative")
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

K14. Write `fib(n)` iteratively (`fib(0)=0`, `fib(1)=1`).

```
def fib(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

K15. Write a recursive `fib_memo(n)` with a memo dictionary, avoiding the mutable-default trap.

```
def fib_memo(n, memo=None):
    if memo is None:
        memo = {}
    if n < 2:
        return n
    if n not in memo:
        memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]
```

K16. Write `char_freq(s)` returning a dict of character counts.

```
def char_freq(s):
    freq = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1
    return freq
```

K17. Write `first_unique(s)` returning the first non-repeating character or `None`.

```
def first_unique(s):
    freq = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1
    for ch in s:
        if freq[ch] == 1:
            return ch
    return None
```

K18. Write `two_sum(nums, target)` returning the two indices, in one pass.

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return seen[target - x], i
        seen[x] = i
    return None
```

K19. Write `second_largest(nums)` (distinct values, no sorting). Return `None` if it does not exist.

```
def second_largest(nums):
    first = second = float("-inf")
    for x in nums:
        if x > first:
            first, second = x, first
        elif first > x > second:
            second = x
    return None if second == float("-inf") else second
```

K20. Write `dedupe(items)` removing duplicates while keeping order.

```
def dedupe(items):
    seen, out = set(), []
    for x in items:
        if x not in seen:
            seen.add(x)
            out.append(x)
    return out
```

K21. Write recursive `flatten(nested)` for arbitrarily nested lists.

```
def flatten(nested):
    flat = []
    for item in nested:
        if isinstance(item, list):
            flat.extend(flatten(item))
        else:
            flat.append(item)
    return flat
```

K22. Write `power(x, n)` recursively with fast exponentiation ($n \geq 0$).

```
def power(x, n):
    if n == 0:
        return 1
    half = power(x, n // 2)
    if n % 2 == 0:
        return half * half
    return half * half * x
```

K23. Write `binary_search(arr, target)` returning the index or -1.

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        if arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

K24. Write `average(*nums)` returning 0 when nothing is passed.

```
def average(*nums):
    return sum(nums) / len(nums) if nums else 0
```

K25. Write `describe(**kwargs)` returning "a=1, b=2" with keys sorted.

```
def describe(**kwargs):
    return ", ".join(f"{k}={v}" for k, v in sorted(kwargs.items()))
```

K26. Fix the mutable default bug: write `append_to(x, lst=None)`.

```
def append_to(x, lst=None):
    if lst is None:
        lst = []
    lst.append(x)
    return lst
```

K27. Write `make_counter()` returning a function that gives 1, 2, 3, ... on successive calls (closure).

```
def make_counter():
    n = 0
    def step():
        nonlocal n
        n += 1
        return n
    return step
```

K28. Sort a list of pairs by second element, ties broken by first element, using a lambda:

`sort_pairs(pairs).`

```
def sort_pairs(pairs):
    return sorted(pairs, key=lambda p: (p[1], p[0]))
```

K29. Write `triangle(n)` returning a right-angled star triangle as one string.

```
def triangle(n):  
    return "\n".join("*" * i for i in range(1, n + 1))
```

K30. Write `rotate_right(lst, k)` rotating a list right by `k` (`k` may exceed the length).

```
def rotate_right(lst, k):  
    if not lst:  
        return []  
    k %= len(lst)  
    return lst[-k:] + lst[:-k] if k else lst[:]
```

Chapter 10 · Common Interview Traps

Skim this list the night before an interview. Each line is a question that has actually been asked in Python screenings.

#	Trap	What really happens	Fix / reason
1	<code>[lambda: i for i in range(3)]</code>	all lambdas return 2	late binding; use <code>lambda i=i: i</code>
2	<code>def f(x, l=[])</code>	list persists across calls	default built once; use <code>None</code>
3	<code>x = f()</code> where <code>f</code> only prints	<code>x</code> is <code>None</code>	use <code>return</code>
4	<code>if x == 1 or 2:</code>	always truthy	<code>x in (1, 2)</code>
5	<code>0 or []</code>	returns <code>[]</code> , not <code>False</code>	<code>or</code> returns an operand
6	removing from a list while looping	skips elements	iterate over a copy / comprehension
7	adding keys to a dict while looping	<code>RuntimeError</code>	loop over <code>list(d)</code>
8	loop variable after <code>for</code>	still exists (last value)	function scope, not block scope
9	<code>for ... else</code>	<code>else</code> runs when <code>NO</code> <code>break</code>	use as "not found" branch
10	<code>count += 1</code> in a function	<code>UnboundLocalError</code>	global <code>count</code> or return value
11	<code>items.append()</code> on a global	works without global	mutation $\neq$ rebinding
12	<code>range(1, n)</code>	excludes <code>n</code>	<code>range(1, n + 1)</code>
13	zip with unequal lengths	truncates silently	<code>zip_longest / strict=True</code>
14	iterating an iterator twice	second pass is empty	<code>enumerate</code> , <code>zip</code> , <code>map</code> , generators are single-use
15	<code>bool("False"), bool([0])</code>	both <code>True</code>	non-empty $\Rightarrow$ truthy
16	<code>0.1 + 0.2 == 0.3</code>	<code>False</code>	<code>math.isclose</code>
17	<code>is</code> vs <code>==</code> on numbers/strings	<code>is</code> may seem to work then fail	use <code>==</code> for values
18	return indented inside the loop	returns after first round	dedent to the loop level
19	<code>a = b = []</code>	both names share ONE list	<code>a, b = [], []</code>
20	<code>a = a.sort()</code>	<code>a</code> becomes <code>None</code>	<code>sort()</code> mutates and returns <code>None</code> ; use <code>sorted(a)</code>
21	<code>print(lst.append(4))</code>	prints <code>None</code>	mutating methods return <code>None</code>
22	<code>def f(a=1, b):</code>	<code>SyntaxError</code>	non-default params first
23	<code>input()</code> compared to <code>int</code>	<code>str</code> vs <code>int</code> : <code>TypeError</code> / wrong logic	<code>int(input())</code>
24	naming a variable <code>list</code> , <code>sum</code> , <code>max</code>	later <code>list(...)</code> breaks (<code>TypeError</code>)	never shadow built-ins
25	<code>10 / 3</code> vs <code>10 // 3</code>	<code>3.333...</code> vs <code>3</code>	<code>//</code> is floor division (rounds toward $-\infty$: <code>-7 // 2 = -4</code>)
26	<code>def f(t=time.time())</code>	same timestamp every call	default evaluated once; use <code>None</code>
27	deep recursion (100 000 levels)	<code>RecursionError</code>	loop / explicit stack; no TCO
28	<code>[[0]*3]*3</code>	3 rows are the SAME list	<code>[[0]*3 for _ in range(3)]</code>

#	Trap	What really happens	Fix / reason
29	<code>a, b = b, a</code> vs <code>a = b; b = a</code>	second version loses <code>a</code>	tuple swap evaluates right side first
30	<code>nonlocal</code> for a global name	<code>SyntaxError</code>	<code>nonlocal</code> needs an enclosing function; use <code>global</code>

Last-minute checklist

- Can I explain LEGB with an example and predict an `UnboundLocalError`?
- Can I fix a mutable default argument from memory?
- Can I say when `for ... else` runs?
- Can I write two-pointer, sliding-window, frequency-map and binary-search loops without notes?
- Can I rewrite a recursive function as a loop, and memoize a recursive one?
- Do I know that `and/or` return operands, and `return` vs `print`?
- Can I list the parameter-order rules including `/` and `*`?

Next in the series → Part 4: deeper data structures (lists, tuples, sets, dictionaries) and the `collections` module, building on the loops and functions you have just mastered.